

Dynamic Programming - Project

Zhihan ZHANG

April 2025

Contents

1	Problem Description and Literature Review	1
1.1	Problem Description	1
1.2	Literature Review	1
2	Baseline Model Formulation	2
2.1	Model Overview	2
2.2	State and Decision Variables	2
2.3	Transition Dynamics	2
2.4	Utility and Objective Function	2
2.5	Bellman Equation	3
2.6	Modeling Assumptions and Rationale	3
3	Algorithms	4
3.1	Method 0: Exact Dynamic Programming (Baseline and Complex Models)	4
3.2	Method 1: Approximate Dynamic Programming via Linear Function Approximation (LFA)	4
3.3	Method 2: Approximate Dynamic Programming via Policy Gradient	5
4	Main Results	6
4.1	Baseline Model Results	6
4.2	Complex Model Results	7
4.3	Continuous-Time Model Insights	7
4.4	Comparative Summary	8

1 Problem Description and Literature Review

1.1 Problem Description

This project estimates how young adults between the ages of 18 and 25 make sequential decisions regarding education, work, delaying and saving college enrollment under financial constraints. In real life, these decisions are shaped by a variety of factors: access to loans, anticipated returns to education, current income needs, and family background. The objective is to find an optimal policy that maximizes expected utility while respecting borrowing constraints and the stochastic nature of wage offers and academic success.

While the problem is inspired by empirical patterns in the U.S. education system by Matthew T. Johnson (2013), this project does not use real-world data. Instead, it builds a structurally motivated simulation model, with assumptions and parameter values based on insights from prior literature and real-world considerations.

In this project, we consider to use both exact dynamic programming (via value function iteration) and approximate dynamic programming techniques (including Bellman with functional approximation, and policy gradient methods). We set up three problems with different model complexity to verify the advantages and disadvantages of exact DP and different ADP methods in different scenarios.

1.2 Literature Review

The problem of dynamic education and labor supply decisions under financial constraints has been extensively studied in structural labor economics and applied dynamic programming. Four key references inspired me from the perspective of topic selection, and some of which describe the approximate DP method, providing a knowledge basis for the methods used in my project.

1. **Matthew T. Johnson (2013) – *Borrowing Constraints, College Enrollment, and Delayed Entry***

This foundational paper builds a dynamic discrete choice model where high school graduates decide whether to enroll in college, work, or delay. Johnson focuses on the role of borrowing constraints in delaying or deterring college enrollment. His model captures forward-looking behavior and calibrates structural parameters using U.S. data. Johnson approximates the conditional expected value (Emax) separately for each action by first evaluating at a subset of states using Monte Carlo simulation and then fitting a polynomial regression for each action-specific value function. This paper inspires the modeling framework of my project, although our approach is simulation-based rather than data-driven.

2. **Michael P. Keane and Kenneth I. Wolpin (2001) – *The Effect of Parental Transfers and Borrowing Constraints on Educational Attainment***

The article aims to explore how parental financial support and borrowing restrictions affect the educational achievement of young men. The model uses the data of the 1979 youth of the National Longitudinal Survey of Labor Market Experience (NLSY) in the United States, covering 11 years of tracking information starting at the age of 16. The model is a dynamic discrete choice model of young people's education/work/saving decisions. Matthew T. Johnson (2013) also stated that their approach builds are based on this article.

3. **Bertsekas and Tsitsiklis (1996) – *Approximate Dynamic Programming via Linear Programming***

Among the approaches they discuss, Approximate Linear Programming (ALP) uses a linear combination of basis functions to approximate the value function while ensuring Bellman inequalities are satisfied. They formulate an Approximate Linear Program (ALP) to solve for the coefficients, ensuring that the approximate value function respects the Bellman inequalities. This work represents a seminal contribution to the field of Approximate Dynamic Programming (ADP) with Linear Value Function Approximation (Linear VFA).

4. **Powell, Warren B. (2011) – *Approximate Dynamic Programming: Solving the Curse of Dimensionality (2nd ed.)*.**

VFA is the main method he explains, and it is divided into: linear approximation (Linear VFA), nonlinear approximation (such as neural network), etc. The book emphasizes how to approximate the value function through designed features, and gives a lot of practical suggestions, such as feature selection, feature scaling, regression technology, etc. Powell also briefly discusses policy search methods, including the idea of policy gradient techniques, which form another branch of approximate DP used in our project.

2 Baseline Model Formulation

2.1 Model Overview

We formulate the educational and career decision-making problem faced by young adults as a finite-horizon discrete-time dynamic program. Each agent begins at age 18 and makes sequential choices annually until age 25. The objective is to maximize the expected utility derived from consumption and behavioral preferences, subject to borrowing constraints and uncertain income.

2.2 State and Decision Variables

At each period t , the agent's state is described by a tuple:

$$\Omega_t = (\text{age}_t, \text{educ}_t, \text{asset}_t, \text{ability}_t),$$

where:

- $\text{age}_t \in \{18, \dots, 25\}$: current age. $\text{educ}_t \in \{0, 1, 2, 3, 4\}$: completed years of college education,
- $\text{asset}_t \in \{-10000, -5000, 0, 5000, 10000\}$: discrete net asset levels,
- $\text{ability}_t \in \{\text{low}, \text{medium}, \text{high}\}$: latent ability type.

The decision at time t is a pair: $\theta_t = (s_t, \text{asset}_{t+1})$

where $s_t \in \{\text{study}, \text{work}, \text{delay}\}$ is the action taken, and asset_{t+1} is the desired asset level for the next period, subject to the constraint $\text{asset}_{t+1} \geq a_{\min} = -10000$.

2.3 Transition Dynamics

State transitions are governed by deterministic and stochastic rules:

- **Asset:** asset_{t+1} is chosen by the agent, subject to borrowing limits.
- **Ability:** evolves according to a first-order Markov process with known transition matrix:

$$P(\text{ability}_{t+1} \mid \text{ability}_t) = \begin{bmatrix} 0.5 & 0.4 & 0.1 \\ 0.1 & 0.6 & 0.3 \\ 0.0 & 0.2 & 0.8 \end{bmatrix}.$$

- **Educ:** If $s_t = \text{study}$, education advances with probability $p(\text{ability}, \text{educ}) = \begin{bmatrix} 0.7 & 0.7 & 0.6 & 0.5 & 0.5 \\ 0.8 & 0.7 & 0.6 & 0.6 & 0.6 \\ 0.9 & 0.8 & 0.7 & 0.7 & 0.7 \end{bmatrix}$,

which depends on ability and current education level. Otherwise, $\text{educ}_{t+1} = \text{educ}_t$.

2.4 Utility and Objective Function

Consumption is defined as:

$$c_t = \text{income}_t - \text{cost}_t - (\text{asset}_{t+1} - \text{asset}_t),$$

where income and shocks only happens when choosing working, depends on education level, ability, and an i.i.d. income shock ε_t with probabilities 0.25, 0.5, 0.25:

$$\text{income}_t = \text{base_wage}(\text{educ}_t) + \text{bonus}(\text{ability}_t) + \varepsilon_t, \quad \varepsilon_t \in \{-3000, 0, 3000\}.$$

Tuition cost is $\text{cost}_t = 3000$ if studying, and zero otherwise. The utility function is CRRA:

$$u(c) = \begin{cases} \frac{c^{1-\rho}}{1-\rho}, & c > 0, \\ -\frac{(-c)^{1-\rho}}{1-\rho}, & c \leq 0, \end{cases} \quad \text{with } \rho = 0.95.$$

Behavioral penalties are imposed as fixed disutility values: -5 for studying, -10 for working, and -2 for delaying. At the terminal age 25, utility is:

$$V_{25}(\text{educ}, \text{asset}) = u(\max(\text{asset}, 1)) + 0.5 \cdot \text{educ}.$$

2.5 Bellman Equation

The agent's value function satisfies the Bellman recurrence:

$$V_t(\Omega_t) = \max_{\theta_t \in \Theta_t} \left\{ E[u(c_t(\theta_t, \Omega_t)) + \text{Penalty}(\theta_t)] \right. \\ \left. + \delta \cdot E[V_{t+1}(\Omega_{t+1}) \mid \Omega_t, \theta_t] \right\},$$

where $\delta = 0.95$ is the discount factor, and the expectation integrates over wage shocks, ability transitions, and educational success probabilities.

2.6 Modeling Assumptions and Rationale

- Ability follows a Markov process to capture heterogeneity and persistence in educational performance. Borrowing is limited to a hard lower bound (-10000), consistent with credit constraints in real-life student financing. Behavioral penalties reflect non-pecuniary costs or preferences associated with each activity (e.g., study stress, job effort).
- The terminal utility design encourages asset accumulation and completion of college education by age 25, while preventing negative consumption at the terminal stage.
- Note that, I use discrete random shock, especially aim to let our exact DP for the baseline model is accurate, that is, calculate the expectation of utility in an exact way, but not with other methods like integration, though integration method for expectation could also be under the exact DP framework.

Compared with the baseline model, we also develop a more realistic complex model. In the complex model, we set continuous normal distribution wage shocks (so we have to use Gauss-hermite quadrature in exact DP). In addition, we also introduced skill (discrete, from 0 to 1, depending on action), work experience (discrete, must work to increase), whether from a high-income family (binary), and the rest of the variables are also enriched from the perspective of range and discreteness.

In addition, I tried to construct a continuous time problem based on the scenario of the complex model. That is, on its basis, I allow an agent to decide its study / work / delay and asset_{t+1} behavior at any time. Please note that our large-state continuous time problem is difficult to get the optimal strategy through Hamilton-Jacobi-Bellman equations, and I will explain the reason later. Therefore, we still based on the way of discretizing time and state, and solving the Bellman equation. Then therefore, we will definitely need a more detailed grid.

In the continuous time problem, we follow the background settings in the complex model, that is, its variables, model structure, constraints and other assumptions. Note that we set 18 to 25 years old as 7 time periods, so $T=7$, and grid it as $N_t=200$, so $dt=7/200$. How to map the complex model to the continuous problem and make them comparable in terms of methods and numerical results? The following example illustrates the detailed treatment of variables in the continuous time problem.

We are based on the following strong assumption: choosing a learning action, even if it is just a few days, can improve the skill or education level in a small amount in proportion to the number of days, and only pay a few days of tuition. Choosing a job is also similar to the concept of daily

settlement. For example, the scale of tuition is 3000 in the complex model, and becomes $3000 * dt$ in the continuous time model, the same thing also applied on the *asset* and *wage*. From the perspective of the transfer function. Take the "educ" variable as an example. In the complex model, once the education is successful, it can be +1. In the continuous time problem, we allow the agent to go to school or drop out at any time, and $+1 * dt$ can be added if the education is successful. Other variables and their transfer logic are handled similarly. This makes our continuous time model and complex model solve almost the same problem without considering time, and also increases comparability.

By adjusting the scale, the complex model and continuous time model are close in macro-numerical results, thus confirming that they solve the same economic problem. Although under different models, the characteristic vectors and parameters in the policy gradient or VFA method are the characteristics of these solution methods themselves, which are the embodiment of their adaptation to specific numerical settings, rather than the change of the essence of the problem.

3 Algorithms

The reason for using different algorithms, is because we gradually enriched the baseline model. As the model becomes more complex, we will find different applicable algorithms.

3.1 Method 0: Exact Dynamic Programming (Baseline and Complex Models)

When the state space remains tractable, we solve the model using exact value function iteration. The Bellman equation is iteratively applied backward from the terminal age $T = 25$ to the initial age $t = 18$. For each feasible state Ω_t and action θ_t , the value function is updated as:

$$V_t(\Omega_t) = \max_{\theta_t \in \Theta_t} \{E[u(c_t(\Omega_t, \theta_t)) + \text{Penalty}(\theta_t)] + \delta \cdot E[V_{t+1}(\Omega_{t+1})]\}.$$

In the baseline model, the finite grids for assets and discrete education levels make full enumeration feasible. In the complex model, however, the state space is expanded by introducing continuous asset levels and additional variables such as work history and parental income. In this case, expectations over continuous shocks are computed using numerical quadrature (e.g., Gauss-Hermite integration).

Although exact DP produces reliable solutions in the baseline model, it suffers from the curse of dimensionality in the complex model even if. The value function table grows exponentially with each additional state dimension, making convergence difficult in practice.

Note: In DP modeling, we refer to actions as θ . In the following methods, adopting conventional reinforcement learning notation, we denote actions as a and model parameters as θ_0 .

3.2 Method 1: Approximate Dynamic Programming via Linear Function Approximation (LFA)

My model complexity and the need for value function approximation (VFA) to address the curse of dimensionality is inspired by Matthew T. Johnson (2013) and Keane and Wolpin (2001). We implement value function approximation (VFA) for both complete and continuous-time model. VFA aims to find a parameterized approximation of the value function $V^\pi(s)$ or $Q^\pi(s, a)$ under a given policy π , or of the optimal value function $V^*(s)$ or $Q^*(s, a)$. A policy can then be derived, for instance, by following the greedy rule $\pi(s) = \arg \max_a Q(s, a)$.

In our case, we use Linear Function Approximation (LFA) to approximate the state value function $V(s)$. The approximated value function takes the form:

$$\hat{V}(s; \theta_V) = \theta_V^\top \phi(s),$$

where $\phi(s)$ is a feature vector of state s , and θ_V is the parameter vector. We train θ_V using Temporal Difference (TD) learning, specifically TD(0). For a transition from state s_t to s_{t+1} after taking action

a_t and receiving reward R_{t+1} , the TD error is defined as:

$$\delta_t = R_{t+1} + \gamma \hat{V}(s_{t+1}; \theta_V) - \hat{V}(s_t; \theta_V).$$

The parameter update rule becomes:

$$\theta_V \leftarrow \theta_V + \alpha \delta_t \phi(s_t),$$

where α is the learning rate. This update moves θ_V in the direction that reduces the squared TD error, progressively making $\hat{V}(s_t; \theta_V)$ closer to the TD target.

In high-dimensional dynamic decision problems, solving for every state-action pair as in traditional grid-based dynamic programming is infeasible. Even after discretization, the state space remains too large. LFA provides a tractable alternative by learning a compact parameterized representation of the value function based on sampled trajectories. This allows generalization to unvisited states and reduces computational complexity to depend primarily on the number of parameters rather than the size of the state space.

In practice, trajectories are generated by interacting with the environment, following a greedy policy derived from the current value function estimate. The parameters are updated via stochastic (semi-)gradient descent to minimize the TD error, thereby approximating the Bellman equation over the observed states.

3.3 Method 2: Approximate Dynamic Programming via Policy Gradient

In the baseline, complex and continuous-time models, we also implement a Policy Gradient (PG) method. Policy Gradient methods directly optimize a parameterized policy $\pi(a | s; \theta)$, which maps states to a distribution over actions, by maximizing the expected cumulative discounted return:

$$J(\theta) = E_{\pi_\theta} \left[\sum_{t=0}^T \gamma^t R_t \right].$$

We parameterize the policy using a softmax over a linear combination of state-action features:

$$\pi(a | s; \theta) = \frac{\exp(\theta^\top \psi(s, a))}{\sum_{a' \in \mathcal{A}(s)} \exp(\theta^\top \psi(s, a'))},$$

where $\psi(s, a)$ is a feature vector combining information from state s and action a , and $\mathcal{A}(s)$ denotes the set of legal actions. The policy parameters θ are updated via stochastic gradient ascent:

$$\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta),$$

where the gradient $\nabla_\theta J(\theta)$ is estimated from sampled trajectories. A standard method is REINFORCE, which approximates the gradient using Monte Carlo returns:

$$\nabla_\theta J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \sum_{t=0}^{T_i-1} \nabla_\theta \log \pi(a_{i,t} | s_{i,t}; \theta) G_{i,t},$$

where $G_{i,t}$ is the discounted return from step t onward in episode i .

For a softmax policy with linear features, the policy gradient has the form:

$$\nabla_\theta \log \pi(a | s; \theta) = \psi(s, a) - E_{a' \sim \pi(\cdot | s; \theta)}[\psi(s, a')],$$

where the expectation is over legal actions weighted by the current policy.

Policy Gradient methods are well-suited for high-dimensional problems, as they optimize a compact parameterized policy without needing to explicitly evaluate all state-action pairs. The policy function $\pi(a | s; \theta)$ generalizes to unseen states and naturally handles stochastic action selection, which can be advantageous in environments with partial observability or when exploration is important.

4 Main Results

In this section, we summarize the key insights obtained from the numerical experiments conducted on the three model variants: the baseline model, the complex discrete model, and the continuous-time extension. The results shows performance trade-offs among solution methods. Owing to space constraints, the discussion of the baseline problem is limited to agents initially aged 18, with an education level of 0, no assets, and classified as high-ability. In the complex and continuous-time problems, the initial conditions further specify a skill level of 0, no work experience, and a background of high parental income.

The two approximate DP methods of either model are performed in random environments during training and simulation. Policy Gradient learns a stochastic policy, so printing the trajectory, it determines the action for each step by sampling from the learned probability distribution through *choose_action*. VFA learns a value function, its corresponding strategy is usually to choose the action that maximizes the value function. This is a greedy strategy, so *best_action* used when printing. For ease of visualization and understanding, the environment used when printing are all deterministic or simplified (for example, using expectation of wage shocks, judging the threshold of education success rate, and transferring the maximum probability of ability, etc.). Besides, all the training epochs = 200, episodes_per_epoch=50 and learning rate = 0.01 for comparison. The printed Exact DP trajectory follows the learned optimal state-action mapping under expected outcomes, a representative path generated in a simplified, deterministic environment for illustration purposes, rather than a realization from a fully stochastic process.

4.1 Baseline Model Results

In the baseline discrete-time model, we compare the performance of exact dynamic programming and policy gradient (ADP). So why we don't use LFA here ? LFA is relatively redundant in this scenario. Given the current limited state space, exact DP can already calculate the exact $V(s)$ and $Q(s, a)$ values for all states and store them in a table. If force LFA in this scenario, we use a linear function approximator to fit the value function that can already be accurately calculated by Exact DP. This is to approximate the function on a discrete lookup table, like fitting a complex function to a bunch of discrete points, and I already know the data of these discrete points accurately.

The powerful VFA can theoretically be very close to the table solution, but makes it less obvious than Policy Gradient when compared as a different method. Therefore, on the baseline model, if I still want to compare Exact DP and ADP, I chose Policy Gradient. As expected, the exact value function iteration converges efficiently and yields the optimal decision paths. For instance, a high-ability agent with no initial assets tends to study in the first one or two periods, followed by a switch to work activity until the terminal age. This behavior is consistent with the model's structure, which rewards early human capital investment and penalizes delayed education.

Baseline Model	Exact DP (0 second)		ADP: Policy Gradient (14 seconds)	
Stage	State	Optimal Action*	State	Optimal Action*
Age 18	(18, 0, 0, 'high')	('study', -5000)	(18, 0, 0, 'high')	('study', -10000)
Age 19	(19, 1, -5000, 'high')	('study', -10000)	(19, 1, -10000, 'high')	('work', -10000)
Age 20	(20, 2, -10000, 'high')	('work', 0)	(20, 1, -10000, 'high')	('work', -10000)
Age 21	(21, 2, 0, 'high')	('delay', -5000)	(21, 1, -10000, 'high')	('work', -10000)
Age 22	(22, 2, -5000, 'high')	('delay', -10000)	(22, 1, -10000, 'high')	('work', -10000)
Age 23	(23, 2, -10000, 'high')	('work', -5000)	(23, 1, -10000, 'high')	('work', -10000)
Age 24	(24, 2, -5000, 'high')	('work', 5000)	(24, 1, -10000, 'high')	('work', -10000)
Age 25 (Terminal)	No action		No action	
$V(s_0)^*$	165.781		149.08	

We see that under the Policy Gradient method, the agent tends to make repetitive actions and extreme asset choices. I think this is because our feature vectors are not rich enough and we did not use the Baseline technique to reduce the large variance of Policy Gradient during training, as Keane and Kenneth I. Wolpin (2001) mentioned in their section 10.7. We implemented a very basic PG method just to show that in low-dimensional conditions, ADP requires a lot of effort to achieve the

optimal solution, both in terms of training time and hyperparameter tuning.

4.2 Complex Model Results

In the extended model with richer state variables as mentioned before, exact DP becomes computationally infeasible due to state space explosion. Even after 3 hours of runtime, no convergence was achieved. In contrast, LFA method and Policy Gradient can rapidly provide results. So we only show the two ADP results.

Complex Model	ADP: Policy Gradient (1 minute)		ADP: LFA (28 seconds)	
Stage	State	Optimal Action*	State	Optimal Action*
Age 18	(18, 0, 0.0, 0.0, 0, 1, 'high')	('study', -20000)	(18, 0, 0.0, 0.0, 0, 1, 'high')	('delay', -5714.29)
Age 19	(19, 1, -20000.0, 0.05, 0, 1, 'high')	('work', -20000)	(19, 0, -6000.0, 0.0, 0, 1, 'high')	('delay', -11428.57)
Age 20	(20, 1, -20000.0, 0.05, 1, 1, 'high')	('work', -20000)	(20, 0, -11600.0, 0.0, 0, 1, 'high')	('delay', -17142.86)
Age 21	(21, 1, -20000.0, 0.05, 2, 1, 'high')	('work', -20000)	(21, 0, -17200.0, 0.0, 0, 1, 'high')	('delay', -20000)
Age 22	(22, 1, -20000.0, 0.05, 3, 1, 'high')	('work', -20000)	(22, 0, -20000.0, 0.0, 0, 1, 'high')	('work', -20000)
Age 23	(23, 1, -20000.0, 0.05, 4, 1, 'high')	('work', -20000)	(23, 0, -20000.0, 0.0, 1, 1, 'high')	('work', -20000)
Age 24	(24, 1, -20000.0, 0.05, 5, 1, 'high')	('work', -15714.29)	(24, 0, -20000.0, 0.0, 2, 1, 'high')	('work', -20000)
Age 25 (Terminal)	No action		No action	
$V(s_0)^*$	148.42		151.32	

We can see that in the VFA method, starting from the initial state, the agent first selected the 'delay' action 4 times, which seems to be to reach the minimum value of the asset. Once the asset minimum value of -20000.0 is reached, the agent selects the 'work' action at all subsequent time points and keeps the asset at -20000.0. From the convergence of the loss function (see the code file), the LFA model has relatively stably fitted the relationship of the Bellman equation on the simulated data. This shows that the training of VFA is successful. However, the loss is small on the states visited during the simulation. For a large number of unvisited states in the model, the LFA approximation may have errors, resulting in suboptimal strategies derived in these states.

As for the PG method, the results are similar to those of LFA in terms of macro patterns, both of which lead to the agent quickly reducing the asset to the minimum, and then maintaining survival and increasing experience by continuous work, while ignoring further education investment. This is their main commonality. Never choosing to learn (education remains at 0) may indicate that the cost of learning (tuition) is too high relative to the benefits (higher wages in the future, but indirectly through skill and educ) under the model parameter settings.

The strategy (Softmax) of Policy Gradient is determined by the feature vector and its weights, and the success of the LFA model also depends on whether the feature vector has enough expressive power to approximate the true optimal value function $V^*(s)$. Therefore, we can say that if a richer (e.g. nonlinear) feature vector relationship is adopted and a more reasonable training process is given, both Policy Gradient and LFA have the potential to find the optimal solution for our medium and high dimensional problems.

4.3 Continuous-Time Model Insights

In the most advanced version, we model decisions in continuous time. Here, exact DP is no longer feasible due to the curse of dimensionality. In the continuous time setting, the core of the dynamic programming problem is the Hamilton-Jacobi-Bellman (HJB) equation. For problems with fewer state variables, we can indeed approximate the HJB equation by numerical methods. Using the finite difference method, a grid is established in the state space, and the partial derivatives are approximated by the function value differences at the grid points, thereby converting the PDE into a large algebraic equation system for solution.

However, we have too many types of variables and their relationships are complex. The number of grid points will increase exponentially, and it is almost impossible to directly solve the HJB equation by constructing and solving such a large matrix. Our method is essentially to solve discrete time, discrete state, and discrete action problems to approximate the original continuous time, continuous state, and continuous action problems. As the grid is encrypted (Nt increases, N_asset increases, etc.), this discrete solution should converge to the solution of the original continuous problem.

Continuous-time Model	ADP: Policy Gradient (59 seconds)	
Stage	State	Optimal Action*
18.00	(18.0, 0.0, 0.0, 0.0, 0.0, 1, 'high')	('delay', -700.0)
18.04	(18.035, 0.0, -700.0, 0.0, 0.0, 1, 'high')	('work', -700.0)
18.07	(18.07, 0.0, -700.0, 0.0, 0.04, 1, 'high')	('work', -700.0)
18.11	(18.105, 0.0, -700.0, 0.0, 0.07, 1, 'high')	('work', -602.0)
18.14	(18.14, 0.0, -602.0, 0.0, 0.11, 1, 'high')	('work', -651.0)
...	...	...
24.83	(24.825, 0.12, -553.0, 0.0, 5.43, 1, 'high')	('delay', -700.0)
24.86	(24.860, 0.12, -700.0, 0.0, 5.43, 1, 'high')	('work', -700.0)
24.90	(24.895, 0.12, -700.0, 0.0, 5.46, 1, 'high')	('work', -700.0)
24.93	(24.930, 0.12, -700.0, 0.0, 5.5, 1, 'high')	('work', -651.0)
24.97	(24.965, 0.12, -651.0, 0.0, 5.53, 1, 'high')	('delay', -700.0)
Age 25 (Terminal)	No action	
$V(s_0)^*$	125.06	

We only show the results from policy gradient here, since LFA method runs for hours so I still don't obtain the results. And the fact that LFA takes too long to run (tens or even hundreds of times longer than PG) may itself make this approach a "not worthwhile" approach. In the case of LFA, the value function needs to approximate the value function over entire state space, while Policy Gradient directly optimizes the policy, and it doesn't need to accurately estimate the value function of the entire space first. We can see that the policy gradient learns a richer set of actions in this higher-dimensional case, no longer blindly selecting a fix action and keeping the asset at the lower limit of the grid. It is gratifying that in 200 time steps, some actions of study are chosen. This shows if this method is given better baseline technology and feature vector construction, and the hyper-parameters during training are optimized, it has the potential to find a solution that maximizes $J(\theta)$ and find an excellent strategy for the agent.

We believe that the decision trends of the agents in the continuous-time model are similar to those in the complex model. Agents almost never choose to learn, prefer to work or delay, and never to save money, and the V^* values in the two models are quite similar. All of this means that in very high-dimensional situations, policy gradient can remain a robust method, assuming it is well trained.

4.4 Comparative Summary

This table summarizes the expected performance of exact and approximate dynamic programming approaches across the three model settings. Please note that, I actually also tried the bellman updates with interpolation, we don't discuss it in this project but put my understanding in this table.

Method \ Model	Baseline model (simple variables, yearly discretization)	Complex model (richer variables, still yearly discretization)	Continuous-time model (richer variables, continuous time)
Exact DP	Effective and accurate. Fast computation and optimal results.	Inapplicable due to high computational cost and state space explosion.	Not suitable for continuous-time settings. Cannot iterate backwards in time.
Approximate DP	Policy gradient shows moderate performance (suboptimal in our case).	1. Policy gradient, LFA and interpolation methods are effective (but could be suboptimal.)	1. Policy gradient is effective and stable. 2. LFA and Naive Bellman + interpolation methods work but take a long time. 3. Ideally we hope to solve HJB with interpolation, however difficult to build huge algebraic equations in our case. ¹

From the perspective of numerical results: in low-dimensional models, exact DP is accurate and fast, while the ADP method is slightly slower and cannot obtain accurate results. In medium and high dimensional cases, we can actually see from the complex model that LFA is slightly better than policy gradient in numerical performance and faster, but due to technical reasons, the results of both methods need to be improved. In very high dimensional cases, although the performance of policy gradient still needs to be improved, it is relatively stable and is a reliable method.

As an extended discussion, I hope to add baseline technology to policy gradient to reduce variance, and use hyperparameter search before training to find the best hyper-parameters. At the same time, for this continuous time problem, policy gradient can be extended to continuous action space, which means that perhaps we don't need to discretize assets.

¹HJB form: $\partial_t V + \max_a \left[u(s, a) + \nabla V^\top f(s, a) + \frac{1}{2} \text{Tr}(\sigma \sigma^\top \nabla^2 V) \right] = 0$